Computer Organization and Assembly Language

Week 02 — Internal Organization, Buses, Registers and the First Assembly Program

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 02

Outline

1. Quick Review
2. Computer Organization & Architecture
3. Why Assembly Language?
4. The Big Picture: Internal Organization
5. Address, Data and Control Buses
6. Memory: Cells, Width and Addresses
7. Registers
8. Fetch–Decode–Execute Cycle
9. Instruction Groups
10. GPR Size and Machine Architecture
11. Example Program in Assembly
12. Endianness
13. Intel Software Developer Manual

Quick Review: What is Done?

- Course outline was discussed in detail.
- History and evolution of microprocessors across generations.
- Evolution of **x86** and **x88** microprocessors.
- Latest **64-bit** architectures.
- Concept of **Multicores**.
- **Hyper-Threading (HT)** and **Simultaneous Multithreading (SMT)**.
- Role of **Floating Point Units (FPUs)**.

Quick Review: Memory Capacity and Organization

- Memory and its organization.
- Capacity calculation and representation.
- Examples of memory capacity:
  - **16K, 32K, 64K, ...**
- Examples of memory organization:
  - **2K** $\times$ **8**, **16K** $\times$ **8**, ...

Remember

Capacity tells you *how much* you can store. Organization tells you *how it is arranged*: number of cells $\times$ width of each cell.

Computer Organization & Architecture

- A computer has two important aspects:
  - **Architecture** — what the computer *looks like* to a programmer.
  - **Organization** — how the computer *is built* internally.
- In simple words:
  - Architecture = Design (*What should it do?*)
  - Organization = Implementation (*How is it done?*)

Computer Architecture (Easy Explanation)

- Deals with the **abstract design** of a computer.
- Includes:
  - Instruction Set (e.g. `ADD`, `SUB`, `MOV`)
  - Data types (8-bit, 16-bit, 32-bit, 64-bit)
  - Memory addressing modes
- Example:
  - A programmer only cares about **instructions and registers**, not the physical wiring.

Computer Organization (Easy Explanation)

- Deals with the **physical implementation**.
- Includes:
  - Control signals
  - How data moves inside the CPU
  - Memory hierarchy (Cache, RAM, ROM)
  - How buses connect CPU, memory and I/O
- Example:
  - Two computers may have the same architecture but different organizations (one faster, one slower).

Architecture vs Organization

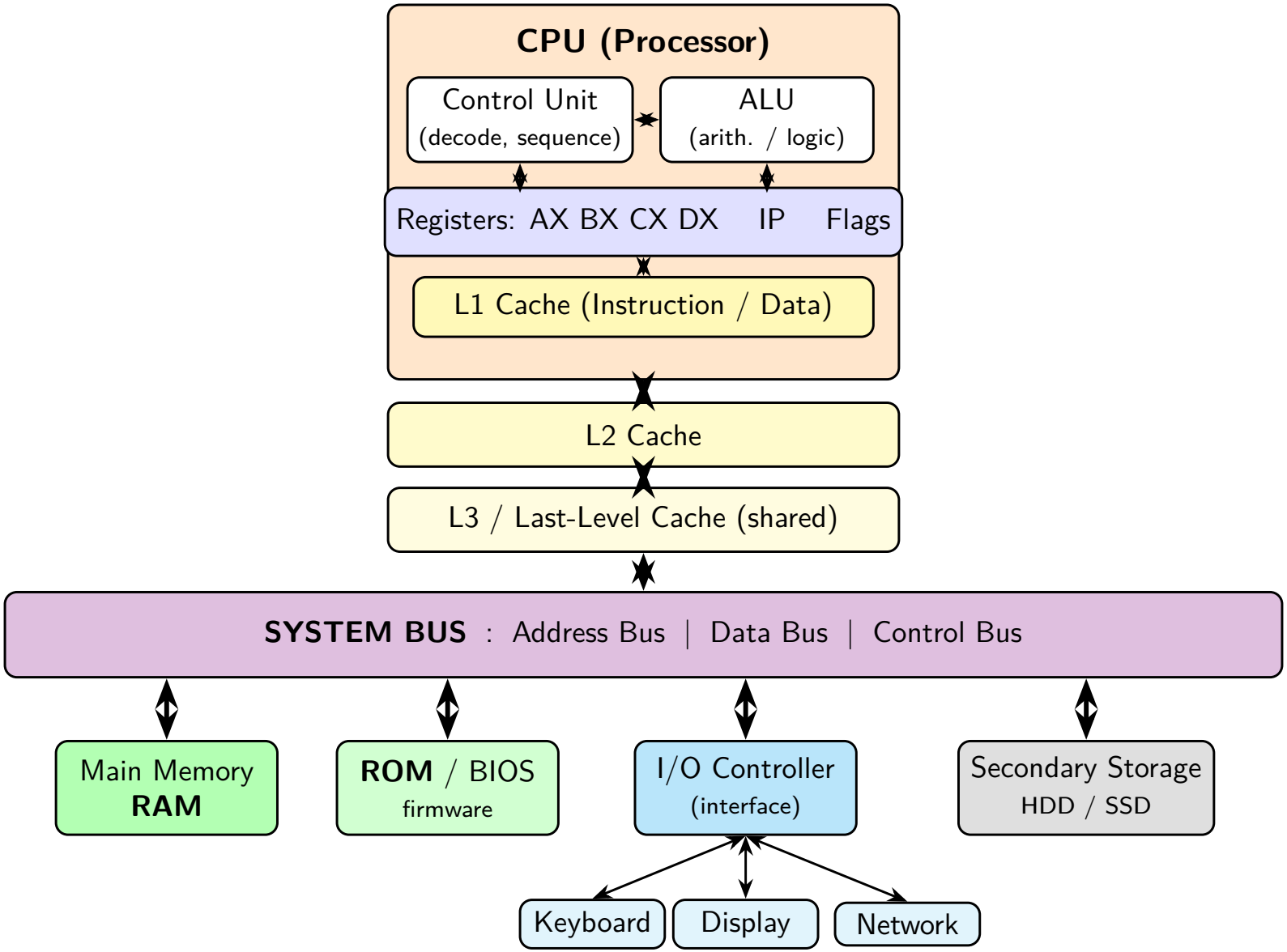
Computer Architecture	**Computer Organization**
What the computer does	How the computer works
Visible to the programmer	Hidden from the programmer
Instruction set, data types, addressing modes	Control signals, memory hierarchy, bus systems
Concerned with *design*	Concerned with *implementation*
Example: x86, ARM	Example: Intel i5 vs i7 (different organizations of the same architecture)

Why Assembly Language?

- **Close to the Hardware** — almost a readable form of machine language (0s and 1s).
- **Efficient** — programs can run faster and use less memory than high-level languages.
- **Direct Hardware Control** — access CPU registers, memory locations and I/O ports directly.
- **Learning Benefit** — shows how computers actually work inside (CPU, memory, buses).
- **Foundation** — strong base for system programming, embedded systems and optimization.
- **Applications:**
  - Performance-critical parts of **Game Development** (graphics, console programming).
  - Low-level routines in **ML / AI accelerators** (GPU kernels, SIMD instructions).

Example: machine code `10110000` vs. assembly `MOV AX, BX`

The Big Picture: A Complete Computer System



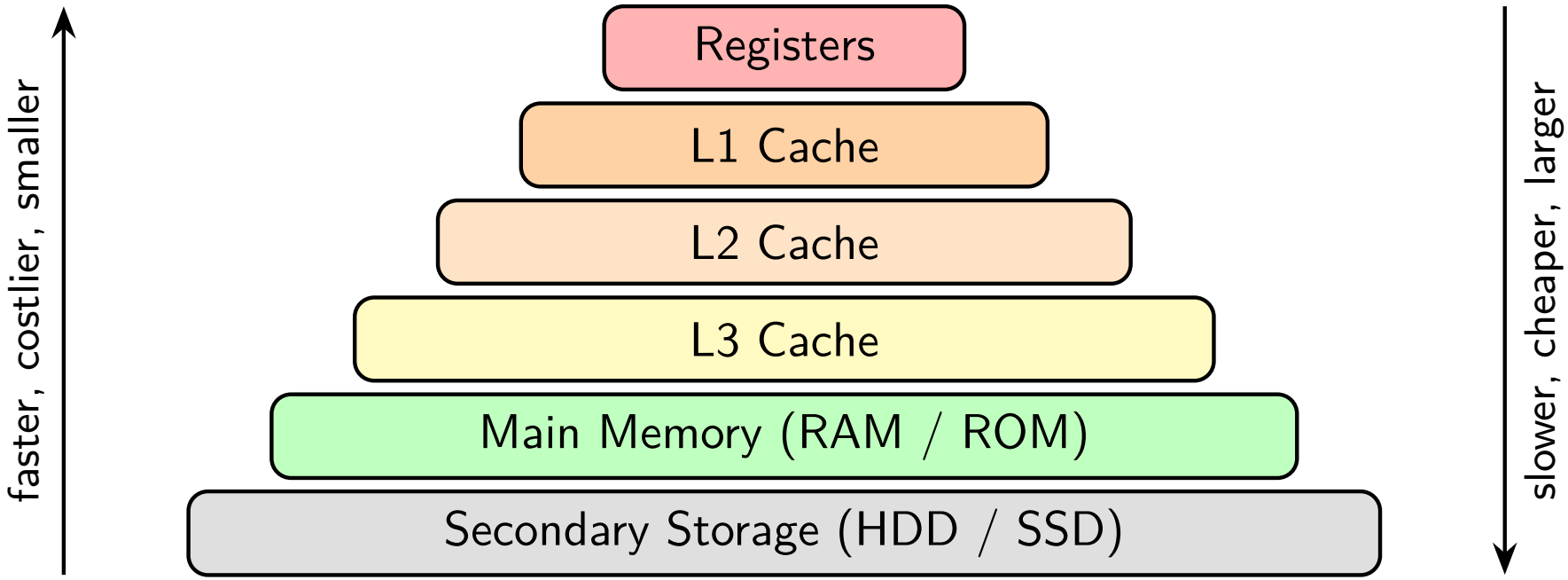
Reading the Big Picture

- A computer system comprises a **processor**, **memory** and **I/O devices**.
- I/O interfaces with the *external* world; memory is the processor's *internal* world.
- The **processor is the core** of this picture and is responsible for performing operations.
- The operation of a computer can be fairly described with processor and memory only — I/O is discussed later in the course.
- The whole working of a computer is:

  the processor performs an operation on data, and that data resides in memory.

- Caches sit *between* the fast processor and the slow memory to hide the speed gap.

Memory Hierarchy at a Glance



- Every level is a compromise between **speed**, **size** and **cost**.

Why Do We Need Buses at All?

- The processor executes operations; the memory contains the data elements.
- This requires a **mechanism for the processor to read that data from memory**.
- "That data" must be described rigorously to the memory, because **memory is a dumb device**.

Analogy: the postman

A postman must be told the *precise address* on the letter to know where the destination is located. He cannot guess. Memory is exactly the same: it must be told precisely *which* cell, and *what* to do with it.

- If we only want to *read* and not *write*, there must also be a way to tell memory which of the two we intend.

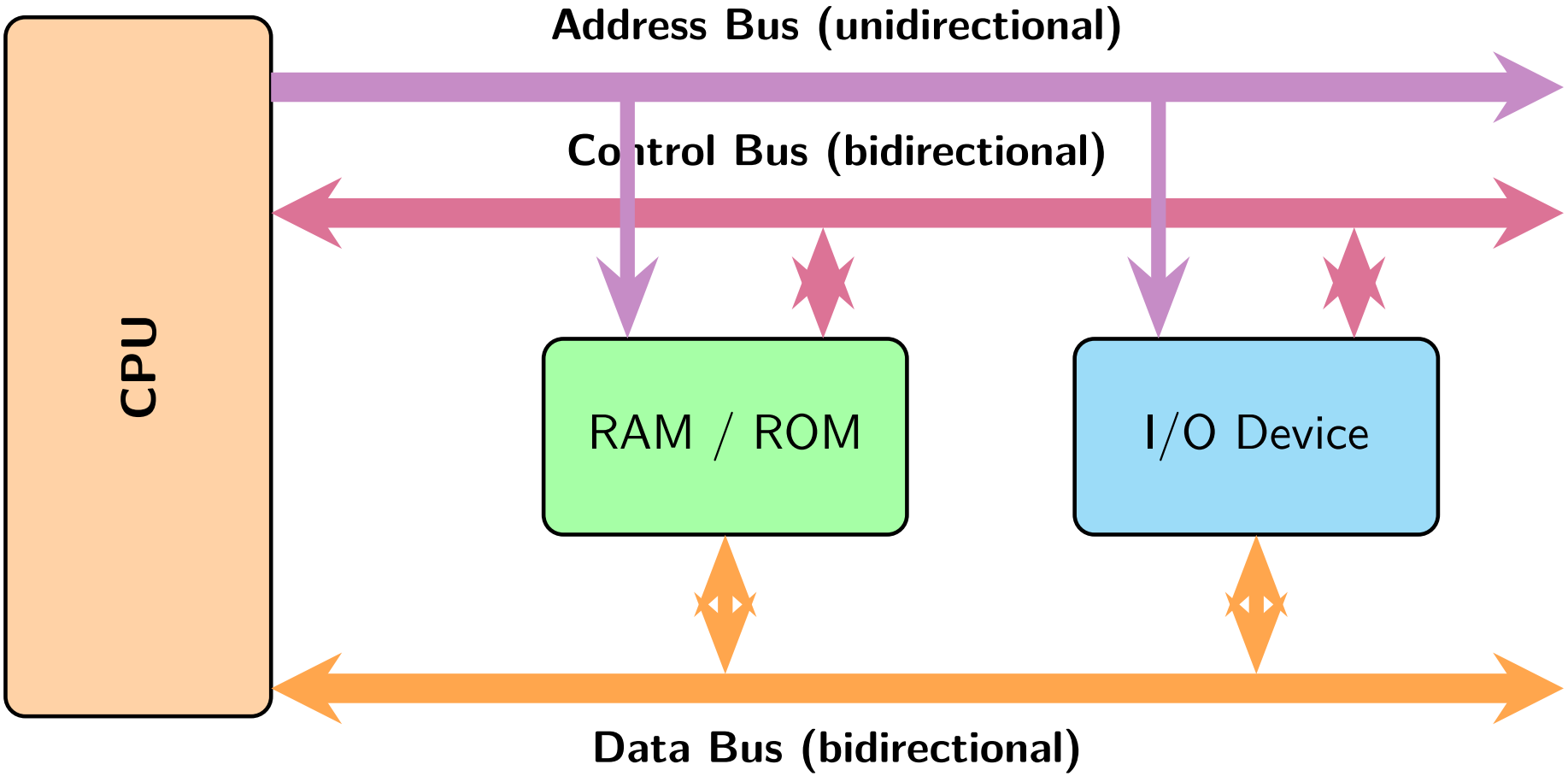
Three Requirements ⇒ Three Buses

1. There must be a mechanism to inform memory that we want to do the **read** (or write) operation. ⇒ Control Bus
2. There must be a mechanism to inform memory **precisely which element** we want to read. ⇒ Address Bus
3. There must be a mechanism to **transfer that data element** from memory to processor. ⇒ Data Bus

Key idea

These three buses are the **eyes, nose and ears of the processor**. It uses them in a *synchronized* manner to perform a meaningful operation. The programmer specifies the operation, but to fulfil it the processor needs the collaboration of other units and peripherals — and that collaboration is made available through the three buses.

Internal Organization: CPU, Memory and the Three Buses



The CPU connects to RAM/ROM and I/O through the Address, Data and Control buses.

The Address Bus

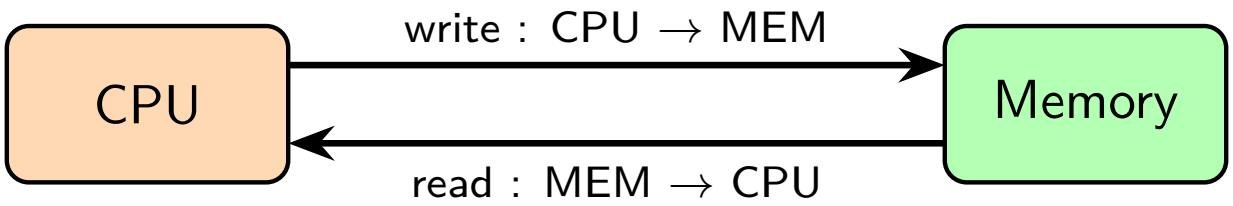
- The group of bits the processor uses to tell memory **which element** to read or write.
- A binary number is generated on the address bus — fifth, seventh, eighth, tenth — *the cell which is needed*.
- The address bus is unidirectional: the address always travels **from processor to memory**.
  - Memory is a dumb device: it cannot predict which element the processor needs at a particular instant of time.
- Width of the address bus $\Rightarrow$ how many cells can be addressed:

$$\text{number of addressable cells} = 2^n \quad (n = \text{address lines})$$

- Example: 16 address lines $\Rightarrow 2^{16} = 64\text{K}$ locations.

The Data Bus

- Used to move the data:
  - from **memory** $\rightarrow$ **processor** in a *read* operation,
  - from **processor** $\rightarrow$ **memory** in a *write* operation.
- Because data moves both ways, the data bus is bidirectional.
- The data bus width limits how much can be moved at a time:
  - In general, **a memory cell cannot be wider than the width of the data bus**.
  - The best and simplest operation requires the **same size of data bus and memory cell width**.



The Control Bus

- A third group of **miscellaneous, independent lines** used for control purposes.
- Example: one line tells the memory whether to perform the **read** or the **write** operation.
- The control bus is **special and relatively complex**, because the different lines that make it up behave differently:
  - some carry information *from* the processor *to* a peripheral,
  - some carry information *from* a peripheral *to* the processor.
- Knowing every signal in detail is unnecessary at this stage, but the general idea of the control bus must be conceived in full.

Control Bus: Coordinating Data Transfer

- The **control bus carries control signals** that coordinate operations between the processor, memory, and I/O devices.
- For a **read** operation, the processor sends a **READ** signal, telling memory that it wants to receive data.
- For a **write** operation, the processor sends a **WRITE** signal, telling memory that it is sending data.
- Thus, the control signals determine **what operation is taking place** and coordinate the direction of data transfer.

Example: Memory Read

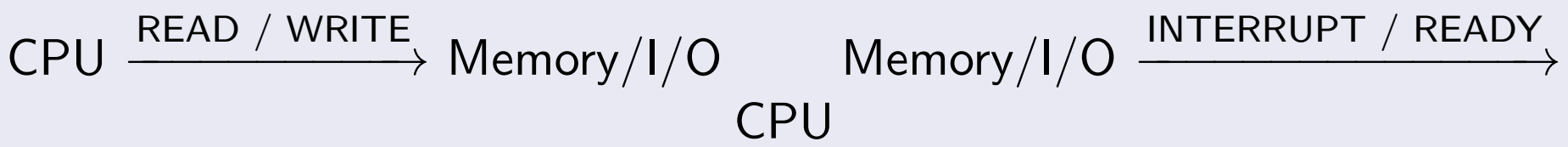
$$\text{CPU} \xrightarrow{\text{Address}} \text{Memory} \qquad \text{CPU} \xrightarrow{\text{READ}} \text{Memory} \qquad \text{Memory} \xrightarrow{\text{Data}} \text{CPU}$$

- The **address bus** specifies *where*, the **data bus** carries *what*, and the **control bus** specifies *what to do and when*.

Then Why Is the Control Bus Bidirectional?

- The control bus consists of **multiple control lines**, and these lines do not all have the same direction.
- Some control signals travel from the processor to memory or I/O devices:
  - **READ**
  - **WRITE**
  - **RESET**
- Other control signals can travel from memory or I/O devices back to the processor:
  - **INTERRUPT** – "I need the CPU's attention."
  - **READY/WAIT** – "I am ready/not ready."
- Therefore, the **control bus is bidirectional as a group**, although each individual control line normally has a specific direction.

Simple View



Summary of the Three Buses

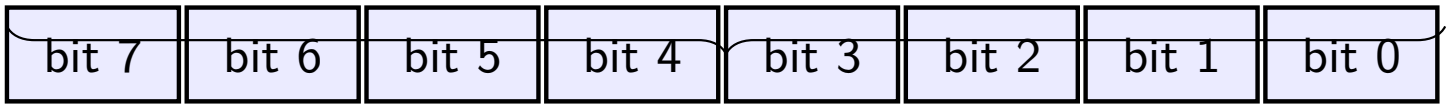
Bus	Direction	Purpose
Address	Unidirectional(CPU $\rightarrow$ MEM)	Selects *which* memory cells or I/O port is involved. Width decides the addressable space 2^n.
Data	Bidirectional	Carries the actual data element; CPU $\rightarrow$ MEM on write, MEM $\rightarrow$ CPU on read.
Control	Bidirectional	Carries intent (read / write), timing and status; peripherals can signal the processor on dedicated lines.

- Same three buses extend naturally to I/O devices — we leave I/O out for simplicity at the moment.

What Exactly Is a Memory Cell?

- A **memory cell** is an n-bit location used to store data — normally 8 bits, also called a **byte**.
- The number of bits in a cell is called the **cell width** (also the *word size of the memory*).
- This need not match the size of a *processor* word, which has other parameters to define it.

cell width = 8 bits (1 byte)



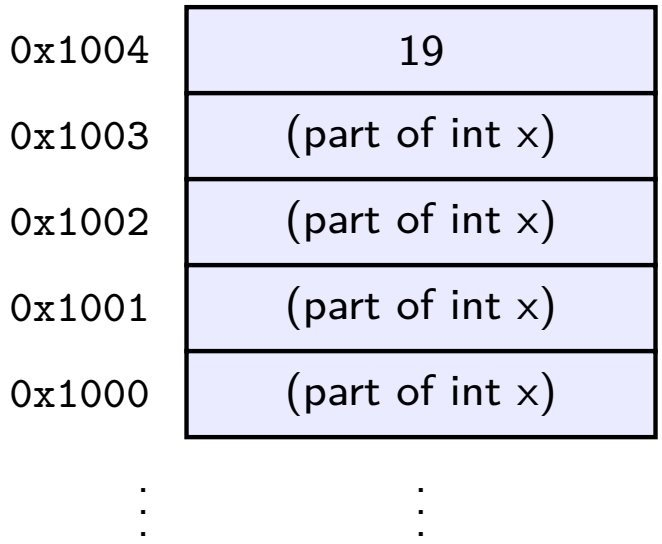
Analogy: the well

A well is completely described by two dimensions: 200 feet deep by 15 feet wide. Memory is the same — **number of cells** $\times$ **cell width** defines it completely, e.g. 2K $\times$ 8, 16K $\times$ 8.

RAM Memory and Its Addresses

- RAM is organized as a large array of memory cells.
- Each cell has a unique **address**.
- Addresses are arranged in a sequence:
  - **High Address** at the top
  - **Low Address** at the bottom

```
int x = 19;
```



RAM storing an `int x = 19`

Two Issues with RAM

- **Speed:** RAM is relatively slow compared to CPU speed.
  - The CPU often *waits* for data to be fetched from RAM.
  - This is why **cache memory** is used to improve performance.
- **Instruction tracking:** the CPU must keep track of instructions and data locations in RAM.
  - It uses the **Program Counter (PC)** to know the next instruction.
  - Extra control logic is required for synchronization.

Why Registers and Cache?

- **Registers:**
  - Located inside the CPU (very close to the processor).
  - Extremely fast compared to cache and RAM.
  - Temporary storage of instructions, addresses and data during execution.
  - Very **expensive** and limited in number.
- **Cache memory:**
  - Sits between CPU and RAM.
  - Stores frequently used instructions/data to reduce CPU waiting time.
  - Faster than RAM, slower than registers.
- **RAM vs Registers:**
  - RAM is very large but extremely slow compared to CPU speed.
  - It cannot be built entirely out of registers. Why? — cost, silicon area, power and physical distance from the ALU.

Why Do We Need Registers?

- The basic purpose of a computer is to perform operations, and **operations need operands**.
- Consider addition: it involves *two* numbers to get their sum.
- But at one instant we can have precisely one address on the address bus, and consequently precisely one element on the data bus.
- As soon as the second operand is selected, the first is no longer there!

Therefore

There are temporary storage places inside the processor called **registers**. One operand is read into a register and added to the other, which is read directly from memory — both are accessible at the same instant, one from *inside* the processor and one from *outside* on the data bus.

Registers: A Scratch Pad Inside the Processor

- The result can then be written to a distinct location, because the operation has completed and a different memory cell can be accessed.
- Sometimes **both** operands are held in registers for efficiency — whatever we do inside the processor is undoubtedly faster than going outside to bring an operand.
- Registers are like a **scratch pad RAM inside the processor**; their operation is very much like normal memory cells:
  - they have precise locations,
  - they remember what is placed inside them.
- They are used whenever we need **more than one data element inside the processor at one time**.
- Memory cells are large in number; registers are very few, and therefore a **scarce and precious resource**.

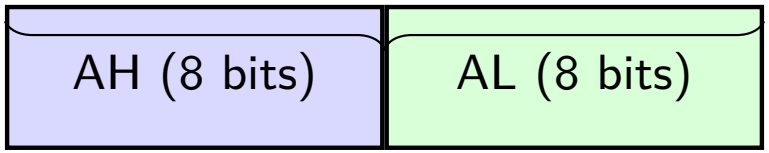
Naming the Registers (Nomenclature)

- Registers are more than one in number, so we must precisely identify or **name** them.
- Some manufacturers *number* them: `r0, r1, r2, ...`
- Others *name* them: `A, B, C, D, ...`
- Still others name them by *function*: `X` for an **index** register.
- This is called the **nomenclature** of a particular architecture.
- Naming is practical exactly because registers are few in number.
- It also tells us that registers have **special functions**: e.g. index registers do not hold data, they hold the **address of data**.

Register Type 1 — The Accumulator

- There is a central register in every processor called the **accumulator**.
- Traditionally **all mathematical and logical operations** are performed on the accumulator.
- The **word size of a processor is defined by the width of its accumulator**: a 32-bit processor has a 32-bit accumulator.

AX — 16-bit accumulator



- In the 16-bit Intel model, `AX` is the accumulator and can also be used as two 8-bit halves `AH` and `AL`.

Register Type 2 — Pointer, Index or Base Register

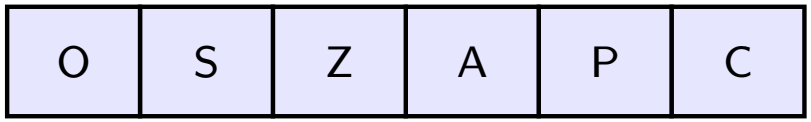
- The name varies by manufacturer, but the distinguishing property is that it does not hold data — it holds the address of data.
- Rationale: consider a `for` loop zeroing an array of ten elements in consecutive memory cells.

```
for (int i = 0; i < 10; i++)
    a[i] = 0; // the ADDRESS changes every iteration
```

- The location to be zeroed **changes every iteration**, so the index register holds the address of the current array element.
- In general: whenever we need access to a memory location whose address is **not known until runtime**, we need an index register. Without it, each iteration would have to be coded explicitly.
- In newer architectures the distinction between accumulator and index registers has become vague — **general purpose registers** can do both, while retaining some specialized behaviours.

Register Type 3 — Flags Register / Program Status Word

- Present in every architecture; 8, 16 or 32 bits like the accumulator.
- Unlike the accumulator it is **meaningless as a unit** — the individual bits carry different meanings.
  - Accumulator bits work *in parallel as a unit*; every bit means the same thing.
  - Flags bits work *independently and individually*; combined, the value is meaningless.



. . . Flags Register

Example: the carry flag

In binary arithmetic a carry can only be 0 or 1, so it fits in a single bit. If a 16-bit number is added to a 16-bit accumulator and the result needs 17 bits, the **17th bit is placed in the carry flag**. Without it the answer is incorrect.

Register Type 4 — Program Counter / Instruction Pointer

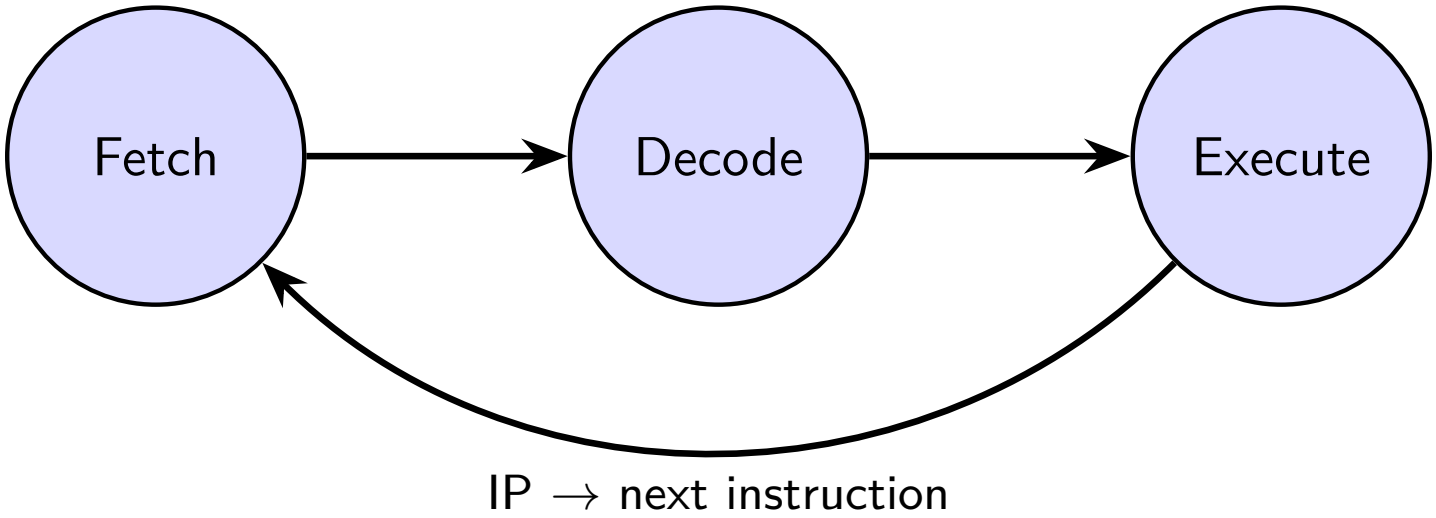
- Everything must translate into a **binary number** for our dumb processor: operands *and* operations themselves.
- A program is defined as "an ordered set of instructions" — *order* is the key part.
  - Instructions have a positional relationship; the whole logic depends on this positioning.
  - If the CPU executed the fifth instruction after the first instead of the second, all our logic is gone.
- A special register ensures this ordering:

Definition

The program counter (instruction pointer) holds the address of the next instruction to be executed.

- The number stored at that address tells the processor which instruction to execute — this number is the **opcode** (e.g. `0xEA`, 255, 152).

CPU Main Objective: Fetch–Decode–Execute Cycle

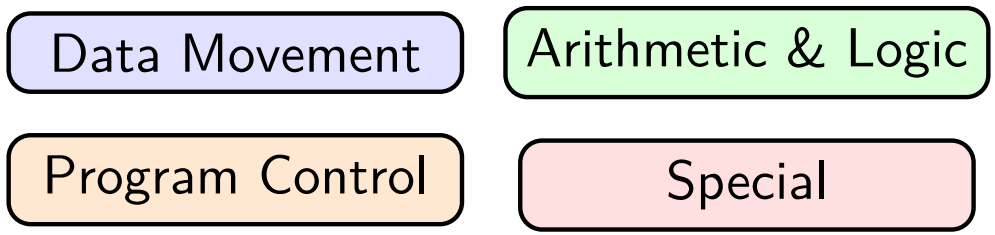


1. **Fetch** — get the instruction from RAM at the address given by the Program Counter.
2. **Decode** — the Control Unit interprets the instruction (which operation must be done?).
3. **Execute** — the CPU carries out the instruction using the ALU, registers and memory.

- One instruction is picked, its operands are read, it is executed; then the next instruction is picked from the new address in the instruction pointer, and so on.
- This cycle repeats continuously while the computer is running.

Instruction Groups: Why Grouping Helps

- Usual opcodes exist in every processor for moving data, arithmetic and logical manipulation, etc.
- Their **mnemonics vary** with the will of the manufacturer: some call data movement `move`, others `load` / `store`.
- But the **basic set of instructions is similar in every processor**.
- Grouping makes learning a new processor quick and easy — just knowing the group an instruction belongs to already tells a lot about it.



Group 1 — Data Movement Instructions

- Used to move data from one place to another.
- Those places can be **registers**, **memory**, or even inside **peripheral devices**.

```
mov ax, bx ; copy contents of BX into AX
lad 1234 ; load from address 1234
```

Group 2 — Arithmetic and Logic Instructions

- Arithmetic: addition, subtraction, multiplication, division.
- Logic: `and`, `or`, `xor`, complement.

```
and ax, 1234 ; logical AND with a constant
add bx, 0534 ; add a constant to BX
add bx, [1200] ; add the DATA PLACED AT address 1200
```

- The **bracketed form** is a more complex variation: it means "add the data placed *at* address 1200".
- Addressing data in memory is a detailed topic and is discussed in the next chapter.

Group 3 — Program Control Instructions

- Instructions normally run one after the other, tied together by the instruction pointer.
- Sometimes we want to **break that implied path**:
  - jump elsewhere if a condition becomes true,
  - or execute a separate block of code and then come back and resume.
- These instructions control program flow by **playing with the instruction pointer**.

```
cmp ax, 0 ; compare AX with 0
jne 1234 ; if not equal, continue at address 1234
```

Group 4 — Special Instructions

- Like "special service commandos": they allow changing specific processor behaviours.
- Used rarely, but certainly used in any meaningful program.

```
cli ; clear the interrupt flag
sti ; set the interrupt flag
```

- `cli` instructs the processor to **close its ears to the outside world** — possibly to do a very important task at hand.
- `sti` restores the normal behaviour.
- Because they change processor behaviour, they form the **special instructions** group.

GPR Size and Machine Architecture

- The size of **General Purpose Registers (GPRs)** defines the architecture of a machine:
  - **16-bit machines** $\rightarrow$ 16-bit registers, smaller address space
  - **32-bit machines** $\rightarrow$ wider registers, larger memory access
  - **64-bit machines** $\rightarrow$ modern standard, huge address space
- **16-bit systems:** simpler environment, typically a single application at a time, no major power or security concerns.
- **Transition to 32-bit:** multiple applications running concurrently $\Rightarrow$ need for **isolation and protection**; security mechanisms became essential.
- **64-bit era:** even more complexity, but improved performance and scalability; security and memory management are integral parts of the design.

Why Focus on 16-bit Architecture First?

- **Learning path:**
  - We start with the **16-bit architecture**.
  - Once 16-bit concepts are clear, moving to 32-bit and 64-bit is much easier.
- **Compatibility issues:**
  - 32-bit applications can often run on 64-bit systems (*backward compatibility*).
  - However, **32-bit and 64-bit applications cannot run directly on 16-bit systems**.
- **Our approach:**
  - We study and practise on the 16-bit model.
  - We use a **virtual machine** (e.g. DOSBox) rather than a physical 16-bit system.
  - This lets 16-bit programs run seamlessly on modern 32-bit or 64-bit computers.

Example Program (Source Code)

```
org 0x0100 ; .COM programs start at offset 0100h

; start of code
mov ax, 5 ; AX = 5
mov bx, 10 ; BX = 10
add ax, bx ; AX = AX + BX -> 15
mov bx, 15 ; BX = 15
add ax, bx ; AX = AX + BX -> 30

mov ax, 0x4c00 ; DOS terminate (AH=4C, AL=00)
int 0x21 ; DOS interrupt: terminate program
```

- **Instruction:** the command itself (`MOV`, `INC`, `ADD`).
- **Operands:** the data or registers the instruction works on. In `MOV AX, 11`: instruction = `MOV`, operands = `AX`, `11`.

Assemble and Produce a Listing with NASM

Build process:

```
nasm c01-01.asm -o c01-01.com
nasm c01-01.asm -l c01-01.lst -o c01-01.com
```

- The first command creates the binary `.com` file.
- The second additionally produces the listing file `c01-01.lst`, which contains:
  - the original source code lines,
  - the instruction addresses,
  - the corresponding opcodes (machine-code bytes).

Run and Debug in DOSBox

Step-by-step procedure:

1. Open DOSBox.
2. Mount your assembly folder:

```
mount X D:\assembly
X:
nasm c01-01.asm -o c01-01.com ; assemble
afd c01-01.com ; debug with AFD
```

- Single-step the program and watch `AX`, `BX`, `IP` and the flags change.
- Match what you see with the opcodes in the `.lst` file.

NASM Listing File (`.lst`)

```
 1 [org 0x0100]
 2
 3 ; start of code
 4
 5 00000000 B80500 mov ax, 5 ; move the constant 5 into register ax
 6 00000003 BB0A00 mov bx, 10
 7
 8 00000006 01D8 add ax, bx ; add value of bx into the value of ax
 9
10 00000008 BB0F00 mov bx, 15 ; move constant 15 into register bx
11 0000000B 01D8 add ax, bx
12
13 0000000D B8004C mov ax, 0x4c00 ; exit ..
14 00000010 CD21 int 0x21 ; .. is what the OS should do for me
```

What to analyze here:

- Instruction addresses in memory (column 2).
- Corresponding opcodes / machine code (column 3).
- Match the opcodes with the debugger's disassembly.

Inspecting Results and Improvement

Debugger checks:

- `AX`/`BX` values:
  - After the first `add` → `AX` = 15 (`0x000F`)
  - After the second `add` → `AX` = 30 (`0x001E`)
- Final state: `mov ax, 0x4c00` overwrites AX.
- Exit code: `AL=0x00` (return code), `AH=0x4C` triggers terminate.

Suggested small fix (preserve the result):

```
mov al, byte ptr ax ; copy low byte of AX into AL
mov ah, 0x4C
int 0x21
```

- Now `AL` carries the result (30 → `0x1E`) as the return code.

Little vs Big Endian (Example with `mov ax,5`)

- **Endianness** = how multi-byte data is stored in memory.
- Register `AX` is 16-bit (2 bytes); `mov ax, 5` $\Rightarrow$ `AX = 0005h`.

Little Endian (Intel)

Address:	1000	1001
Data:	05	00

Big Endian

Address:	1000	1001
Data:	00	05

- **Little Endian:** low-order byte at the lower address.
- **Big Endian:** high-order byte at the lower address.

Note

Intel x86 processors **always** use Little Endian.

Intel Software Developer Manual

- The **Intel Software Developer's Manual (SDM)** is the official reference for Intel processors.
- Covers instruction sets and system programming details for **8-bit, 16-bit, 32-bit and 64-bit architectures**.
- Essential for understanding:
  - assembly instructions,
  - CPU modes and registers,
  - system-level programming.
- **Useful references (check in the SDM):** pages **55, 57, 67, 76**.

Link: Intel® 64 and IA-32 Architectures Software Developer Manuals.

Summary of Week 02

- **Architecture** is what the programmer sees; **organization** is how it is built.
- The processor talks to memory and I/O through three buses: **address** (unidirectional), **data** (bidirectional) and **control** (bidirectional).
- Memory is defined by **number of cells** $\times$ **cell width**; a cell cannot be wider than the data bus.
- **Registers** exist because only one operand can be on the data bus at a time: accumulator, index/pointer, flags, and the instruction pointer.
- Instructions reduce to **opcodes**; mnemonics + assembler make them human-readable.
- Four instruction groups: data movement, arithmetic/logic, program control, special.
- Intel is **little endian**; we practise on the 16-bit model inside DOSBox with NASM and AFD.

Thank You

Any Questions?

"Once you understand the brain of the CPU, everything else in computer systems becomes much easier to grasp."

"If you understand the CPU, you understand the computer."